

Análisis preliminar de herramientas para Vibecoding aplicadas al desarrollo de software

*Minaudo Lucas Agustín, Polito Thiago Martin, Ghiano Gonzalo Agustín, De Luca Leonel
Maximiliano, Gomez Matias Alejandro.
Universidad Nacional de La Matanza
Buenos Aires, Argentina
{gghiano, lminaud, tpolito, leodeluca, matiasalgomez}@alumno.unlam.edu.ar*

1. Introducción

El objetivo del siguiente trabajo es analizar la generación de código de dos de los últimos modelos de LLM especializados para código que se encuentran presentes en el mercado, mediante la elaboración de un algoritmo de grafos concurrente a partir de la técnica de *Vibe Coding*. Se utilizarán métricas para realizar comparaciones entre estas y formar parte de un estudio mayor conformado por la cátedra de Programación Concurrente de la Universidad Nacional de la Matanza.

En el contexto del *Vibe Coding*, el desarrollo mediante el uso de distintas inteligencias artificiales posibilita la exploración de múltiples variantes concurrentes del mismo problema, permitiendo que cada I.A. proponga sus propias estrategias de sincronización, distribución de carga o acceso compartido, que luego son evaluadas con base en métricas cuantitativas de desempeño.

Definimos la concurrencia como la capacidad de un sistema para ejecutar múltiples tareas que se solapan en el tiempo, permitiendo un uso más eficiente de los recursos disponibles, coordinando y comunicando hilos y procesos [1]. A diferencia del paralelismo que busca la ejecución simultánea en varios núcleos de procesamiento.

El caso de estudio será la generación de un algoritmo de búsqueda en grafos Breadth-first search, comúnmente notado como BFS, cuyo objetivo es recorrer los nodos de un grafo nivel por nivel. Este algoritmo es utilizado en una variedad de diversos

estudios sobre concurrencia [2] por lo que conforma un ejemplo conocido para tomar métricas y compararlas.

Los modelos elegidos son Claude Sonnet 4.5 de la empresa Anthropic en su versión de chat a través de la plataforma claude.ai, que fue lanzado el 29 de septiembre de 2025; y GPT 5-Codex a través del entorno de desarrollo Cursor, que se habilitó al público el 15 de septiembre de 2025. Siendo este análisis una comparativa de dos herramientas de vanguardia en el momento de la realización de este documento.

Para la comparación entre los dos algoritmos desarrollados por cada una de las IAs se va a utilizar distintas métricas cuantitativas de desempeño, las cuales permiten cuantificar el comportamiento del sistema en distintos aspectos. Gracias a estas se logrará evaluar algunos de estos aspectos, tanto técnicos como de entendimiento y funcionalidad.

Las métricas a utilizar para la comparación entre los dos algoritmos generados van a ser las siguientes:

Para *Métricas de Calidad y Precisión del Código* se usarán:

- Exactitud Funcional: Se evalúa el nivel de cumplimiento del código para las pautas presentadas, analizando uno por uno los requerimientos detallados solicitados en el prompt técnico.
- Legibilidad y Mantenibilidad del Código: Se analiza claridad, organización y consistencia del código
- Cantidad de Refinamientos: Se contabilizarán la cantidad o ajustes necesarios para corregir el

código o cumplir con los requerimientos solicitados en el prompt técnico.

Para *Métricas de Eficiencia y Rendimiento* se usarán:

- Tiempo de Finalización de la Tarea: Se contabilizará el tiempo total que tarda cada algoritmo en realizar la búsqueda BFS en casos de prueba predefinidos, permitiendo analizar la agilidad y velocidad de cada uno.

2. Desarrollo

El código debe ser generado en el lenguaje de programación Java 17 para plataforma Windows, y debe abarcar la codificación de una clase Grafo que implementa nodos y listas de adyacencia, una implementación del algoritmo BFS que utilice mecanismos de concurrencia y sincronización y lotes de prueba para poder corroborar efectivamente la ejecución del programa.

Para la experiencia utilizamos un prompt técnico estructurado correspondiente a la clasificación Prompt Pattern, particularmente el subtipo Input Semantics.

Este prompt combina características de los patrones Meta Language Creation (secciones fijas para reducir ambigüedades) y Output Customization (restricciones específicas sobre la salida esperada), acercándose al patrón Template Pattern.

LENGUAJE DE PROGRAMACIÓN:

- Java 17

PLATAFORMA:

- Windows

FUNCIONALIDADES A IMPLEMENTAR:

- Desarrollar un proyecto en Java 17 para poder estudiar un algoritmo de grafos BFS que aplique la concurrencia.

REQUISITOS ESPECÍFICOS:

- Implementar una clase Grafo, con nodos y listas de adyacencia.
- Diseñar y construir un algoritmo de búsqueda BFS que utilice herramientas de concurrencia propias del lenguaje Java, prestando especial atención a las herramientas de sincronismo y las variables de acceso compartido.
- Dos lotes de pruebas que permitan verificar el correcto funcionamiento del algoritmo BFS, imprimiendo el recorrido del grafo en el orden esperado por el algoritmo.

El código generado por la herramienta Claude Sonnet 4.5 es accesible a través del siguiente repositorio [Repositorio BFS Concurrente Claude Sonnet](#).

El mismo fue llevado a un proyecto creado manualmente utilizando el IDE Eclipse y ejecutado con JAVA 17 a través del mismo.

Como código generado en el proyecto tenemos la clase Grafo que se encarga de almacenar el número de vértices y la lista de adyacencias entre vecinos. Y tiene la lógica encargada del manejo del grafo en sí, como agregar elementos o imprimirlo. Luego tenemos la clase BFSConcurrente que se encarga del manejo del grafo de manera concurrente, la generación y manejo de hilos y accesos a zonas compartidas. Para los casos de prueba simplemente generó grafos de manera ordinaria y ejecutó el algoritmo sobre estos mediante una clase Main.

Por otro lado, utilizamos el IDE Cursor, que posee integración nativa con agentes de código, donde utilizamos el modelo GPT 5-Codex para generar el proyecto completo ejecutable en el mismo entorno de desarrollo a través de la herramienta de gestión de proyectos Maven.

El código de dicho proyecto es accesible a través del siguiente repositorio [Repositorio BFS Concurrente GPT 5-Codex](#).

En este caso, las clases que podemos encontrar son similares a las generadas por Claude, la clase Graph se encarga de todo lo relacionado al manejo del grafo, y ConcurrentBreadthFirstSearch realiza el recorrido del grafo de manera concurrente.

A diferencia de Claude, GPT generó los tests utilizando Junit5 como framework de pruebas unitarias, validando la salida esperada de forma automatizada.

3. Resultados

Legibilidad y Mantenibilidad del Código Claude Sonnet 4.5		
Criterio	Nivel (1-5)	Comentario
Estructura y organización del código	3	Se podría mejorar la modularidad separando ciertas funciones en una clase aparte

Nomenclatura de variables y funciones	5	nombres de variables y funciones significativos
Indentación y formato	5	Indentación y formato consistente en todo el código
Comentarios y documentación	5	El chat generó documentación sobre el código con todos los puntos importantes para el entendimiento del código
Legibilidad de métodos y funciones	4	Funciones legibles y entendibles
Consistencia y mantenibilidad general	5	Código entendible de manera sencilla y concisa

Legibilidad de métodos y funciones	2	Funciones utilizan métodos complejos sin explicarlos
Consistencia y mantenibilidad general	3	Mezcla idiomas en comentarios y código. Poco mantenible por los problemas mencionados anteriormente

En cuanto a la cantidad de refinamientos, ambos modelos fueron muy buenos, en el primer intento ambos llegaron al resultado esperado. La diferencia fue que GPT al estar en un IDE generó el proyecto completo incluyendo su estructura, cosa que Claude al ser web no puede generarlo, por lo que se le tuvo que preguntar qué estructura era necesario realizar.

Por otro lado, en cuanto a la exactitud funcional, Claude cumplió a la perfección con todos los requisitos solicitados, generando la clase solicitada, el algoritmo y los lotes con su correspondiente impresión. En cambio GPT generó la clase y el algoritmo más los lotes, pero no cumplió con la impresión para la visualización del recorrido del algoritmo.

Como métrica de eficiencia y rendimiento, se elaboró un caso prueba de un grafo comprendido por 5 niveles, comenzando por un nodo padre, conectado a 20 nodos hijo, para los cuales cada uno tendrá otros 20 nodos hijo y esto se repetirá a lo largo de los 5 niveles; llegando a un total de $1 + 20 + 400 + 8000 + 160000 = 168421$ nodos totales. Luego, se aplicó BFS 5 veces sobre el nodo y se tomó el promedio de la duración de la ejecución del algoritmo medido en milisegundos.

Resultado para GPT 5-Codex:

Duración promedio BFS (ms): 104.13512

Resultado para Claude Sonnet 4.5:

Duración promedio BFS (5 ejecuciones): 71.17014 ms

Total de nodos visitados: 168421

Threads utilizados: 12

Debemos aclarar que para la prueba de rendimiento en este modelo, hemos desactivado algunas salidas por pantalla que realizaba el algoritmo para mostrar su procesamiento, ya que la competencia por este recurso estaba desfigurando los resultados reales en gran medida.

Legibilidad y Mantenibilidad del Código GPT 5-Codex		
Criterio	Nivel (1-5)	Comentario
Estructura y organización del código	4	Clases bien moduladas
Nomenclatura de variables y funciones	5	nombres de variables y funciones significativos
Indentación y formato	5	Indentación y formato consistente en todo el código
Comentarios y documentación	2	Descripción breve de las funciones principales, ausencia de documentación

4. Conclusiones

Ambos algoritmos tuvieron resultados más que satisfactorios desde la primera iteración, utilizando mecanismos de concurrencia de hilos para poder procesar de forma más eficiente un mismo nivel y garantizando una ejecución segura a través de tipos nativos concurrentes de Java como `ConcurrentHashMap`, `synchronizedList`, `ConcurrentLinkedQueue`, entre otros.

En ambos casos los modelos infirieron la forma correcta de implementar concurrencia de forma segura y sin romper la naturaleza propia del algoritmo BFS, ya que se utilizan hilos para procesar un mismo nivel de forma concurrente pero no se pasa al siguiente nivel hasta que se termina de forma completa el actual, además de asegurar que no se visite más de una vez a un nodo, a pesar de ejecutarse en distintos hilos y tener aristas que forman bucles en el grafo. En resumen, la implementación del algoritmo fue exitosa, utilizando de guía un prompt con instrucciones sobre las estructuras que debían formar el grafo, pero no la forma en que debía aplicarse la concurrencia en el algoritmo, dejando esta tarea a los propios modelos.

Ambos modelos generaron lotes de prueba claros y comprobables como les fue solicitado, pero además, Claude Sonnet 4.5 generó documentos Markdown explicando los mecanismos de concurrencia utilizados en la realización del algoritmo BFS, permitiéndonos una comprensión más profunda y rápida de su funcionamiento.

Por otra parte, la facilidad de utilizar GPT 5-Codex en la plataforma Cursor se vio con claridad a la hora de necesitar un proyecto estructurado y listo para ejecutar desde la primera iteración, y además, cuando debimos realizar posteriores modificaciones como generar algunas pruebas de rendimiento, ya que el IDE provee a los modelos el contexto necesario para ayudar al mismo a realizar cambios sobre el código, sin necesidad de enviarle fragmentos de código de forma manual.

En síntesis, la integración de la inteligencia artificial en el desarrollo concurrente representa un avance prometedor pero complementario, que no sustituye la comprensión teórica y analítica del programador, sino que la amplifica. El futuro del desarrollo asistido por IA se vislumbra en la colaboración sinérgica entre la creatividad humana y la eficiencia algorítmica, donde ambas partes contribuyen a construir sistemas más potentes, legibles y escalables.

5. Referencias

- [1] Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.
- [2] Juneja K, Khurana D. (2024). A Multithreaded-BFS Algorithm for Optimizing the Graph Computation in Multicore Processing System. *Procedia Computer Science*. <https://doi.org/10.1016/j.procs.2024.03.203>